



Software Engineering Department

Braude College of Engineering

Capstone Project – Phase B (61999)

DeMaestro:

[Autonomous Multi-Agent System for Full-Stack Web Application Synthesis](#)

CODE: 26-1-D-25

By:

Owise Zoubi: owisezoubi@gmail.com

Mohamad Atamneh: Mohamad.atamneh@e.braude.ac.il

Advisor:

Dr. Natali Levi

Link to GitHub

<https://github.com/owisezoubi/DeMaestro>

Project URL:

<https://de-maestro-code-53sp.vercel.app/>

Project Code:

<https://github.com/owisezoubi/DeMaestro-Code>

Abstract.....	2
1.0 Introduction.....	2
2.0 Background and Related Work.....	4
2.1 AI-Based Code Assistants.....	4
2.2 Natural Language–Driven Application Generation.....	4
2.3 No-Code and Low-Code Development Platforms.....	5
2.4 Multi-Agent LLM Systems.....	5
3.0 General Description.....	5
3.1 System Objectives.....	5
3.2 Target User Identification.....	6
3.3 The Agent Architecture: Centralized Orchestrator Multi-Agent Pattern.....	6
4.0 Development Process.....	8
4.1 Phase 1: Research and Planning.....	8
4.2 Phase 2: Architecture Design.....	8
4.3 Phase 3: The Development Phase.....	9
4.4 Phase 4: Integration and Testing.....	10
4.5 Phase 5: Refinement and Optimization.....	10
5.0 System Architecture.....	10
5.1 System Package Diagram.....	11
5.1.1 The Frontend Layer (Web Client).....	11
5.1.2 The Backend Orchestration Layer.....	12
5.1.3 The Data and Persistence Layer.....	12
5.1.4 The AI Agent Engine Layer.....	12
5.1.5 The Subprocess Execution and Cloud Deployment Layer.....	13
6.0 Challenges and Obstacles.....	15
6.1 Enforcing Output Determinism from Generative AI.....	15
6.2 Host-Safe Execution via Subprocess Virtualization.....	15
6.3 The Debugger Strategy Ladder.....	16
6.4 Serverless Cloud Deployment via Vercel.....	17
6.5 Lessons Learned.....	17
7.0 Project Metrics.....	17
7.1 System Performance and Generation Time.....	18
7.2 Code Reliability and Validity Rate.....	18
7.3 Usability and Interface Effectiveness.....	18
References.....	19
Appendices.....	20
Appendix A – User Guide.....	20
A.1 Introduction.....	20
A.2 Operating Instructions.....	20
A.3 Navigation and Workspace Management.....	25
Appendix B – Maintenance Guide.....	25
B.1 Introduction.....	25
B.2 System Requirements.....	26
B.3 Configuration and API Credentials.....	26
B.4 Orchestration Backend Setup (FastAPI).....	27
B.5 Sub-System Maintenance Guidelines.....	27

Abstract

Developing a full-stack web application requires clear requirements and careful coordination between the frontend, backend, and database. Users often describe their ideas in informal text or documents that are incomplete or unclear, which makes it hard to turn them into a working application. Existing AI-based development tools focus mostly on code generation or visual building. They usually assume the requirements are already well-defined, provide limited validation, and do not clearly show how user input becomes the final system. This project presents DeMaestro, a system that generates a complete and runnable full-stack web application directly from user requirements. The system first analyzes and clarifies the input, presents a structured summary for user approval, and only then generates the design and source code. The generated project is verified inside an isolated subprocess sandbox and then deployed to a serverless cloud endpoint (Vercel + a per-project Neon Postgres database), producing both a live URL and a downloadable ZIP that the user can extend further.

1.0 Introduction

Developing a modern web application is a complex process that includes several stages and different areas of knowledge, such as requirements analysis, system design, implementation, and validation [2][15]. To build a full-stack web application, a developer must understand user requirements, design the system architecture, implement the frontend and backend, design a database, and make sure that the application runs correctly [2][15]. Experienced development teams manage this process with established software engineering practices, methodologies, and lifecycle models [2][15]. However, for individuals, students, and small organizations, turning an idea into a working application is difficult, time-consuming, and expensive [2][15].

One of the main challenges in software development is correctly understanding and translating user requirements into technical solutions. Users usually describe their needs using natural language, documents, or informal explanations, and these descriptions are often incomplete, unclear, or ambiguous [2][8]. Traditional software engineering solves this problem through structured requirements engineering, where analysts manually convert user input into formal specifications before development begins [2][8]. When requirements are not clearly defined, the final software may not fully meet user expectations, even if the code itself is technically correct [2].

In recent years, artificial intelligence tools have been introduced to support software development. AI-based coding assistants such as GitHub Copilot [3] and Amazon CodeWhisperer [4] help developers by suggesting code during implementation. Other platforms, such as Replit [6], let users generate or modify applications using natural language prompts. In addition, no-code and low-code platforms like Bubble [9] allow users to create applications using visual tools without writing code. These tools aim to reduce development time and lower the technical barrier for building applications [5][10].

Although these solutions provide useful support, they focus mainly on code generation instead of requirements-first development. Studies show that most AI coding assistants assume the user already knows what to build and how the system should be structured, and therefore do not address requirement analysis or validation [5][7]. Application generation platforms often produce code in one step, without clearly explaining how

user requirements are interpreted, validated, or confirmed [7][11]. No-code platforms simplify development but limit flexibility and usually do not give full ownership or transparency of the produced code, which restricts customization and scalability [9][10].

Another limitation of current solutions is the lack of validation and accountability throughout the development process. Research in requirements engineering and AI-assisted programming shows that there is often no clear traceability between the original user requirements and the generated implementation [8][11]. When generated applications fail to meet user expectations, existing tools rarely explain which requirement was misunderstood or missing [5][11]. As a result, users have to manually review, debug, and extend the generated code, which requires technical expertise and reduces the benefit of automation [5][11].

To address these limitations, this project introduces DeMaestro, a role-based system that generates complete and runnable full-stack web applications directly from user requirements. DeMaestro is built as a multi-agent system with specialized roles drawn from two foundational LLM providers: Google Gemini for analytical reasoning (requirement extraction, clarification, blueprint), and Anthropic Claude for code synthesis and repair (architect, generator, tester, debugger, verifier) [14]. These roles operate sequentially within a phase-based workflow, where the output of each phase is finalized before the next phase begins.

DeMaestro follows a requirements-first approach inspired by established requirements engineering practices [2][8]. User input is first transformed into structured requirements and presented back to the user as a Requirements Summary Document for explicit approval. Only after this approval does the system proceed to blueprint and code generation. This approach improves requirements consistency, traceability, and transparency, and reduces misunderstandings in later development stages [2][8].

Based on the approved requirements, the system generates a full source code project that includes frontend, backend, and database components. The generated project is verified inside a local subprocess sandbox and then deployed to a serverless cloud endpoint (Vercel + a per-project Neon Postgres database), so the user gets both a live URL and a downloadable ZIP archive [15].

The goal of DeMaestro is not to generate production-ready software, but to provide a solid and executable starting point for further development. By producing consistent architecture, complete source code, and intermediate artifacts that improve transparency and traceability, DeMaestro reduces the gap between user ideas and initial software implementations while staying feasible within the scope of a bachelor-level software engineering project [2][15].

This book is organized as follows. Chapter 1 presents the background and motivation. Chapter 2 reviews existing solutions and related work in AI-assisted software development. Chapter 3 gives a general description of DeMaestro and its agent architecture. Chapter 4 details the five-phase development process. Chapter 5 presents the system architecture. Chapter 6 discusses the technical challenges we faced and the engineering solutions we used. Chapter 7 presents empirical evaluation and project metrics, followed by the references.

2.0 Background and Related Work

Translating unstructured user intent into reliable software systems is a fundamental challenge in software engineering, and one that is increasingly being addressed with Artificial Intelligence (AI). Large Language Models (LLMs) have introduced powerful new frameworks for code generation and system design, but moving from specification to full implementation is still complex. This section reviews the existing landscape of AI-assisted software development, including real-time code assistants, natural language–driven application generators, no-code and low-code platforms, and multi-agent LLM systems. By looking at the strengths and limitations of these tools — especially around requirements engineering, traceability, and user validation — this review shows the enduring semantic gap between human intent and executable code, which motivates the DeMaestro project.

2.1 AI-Based Code Assistants

AI-based code assistants are one of the most common uses of LLMs in software engineering. Tools such as GitHub Copilot [3] and Amazon CodeWhisperer [4] are integrated into development environments and provide real-time code suggestions. These systems can generate functions, complete code blocks, and recommend common programming patterns.

The main advantage of code assistants is that they speed up the coding process and reduce repetitive tasks. Studies show that such tools can improve developer productivity, especially for routine programming work [5]. However, code assistants are assistive tools, not autonomous systems. They assume that the developer has already defined the system requirements and architecture.

From a software engineering point of view, code assistants do not address the full challenge of translating user requirements into complete applications. They work at the code level and do not provide mechanisms for requirement modeling, user confirmation, traceability, or full-stack integration guarantees [2][8].

2.2 Natural Language–Driven Application Generation

Some platforms try to go beyond code assistance by generating applications directly from natural language descriptions. These systems let users describe an application in text and receive a working program as output. Platforms such as Replit provide features that support this kind of interaction [6].

Although these systems show the potential of LLMs to automate higher-level development tasks, the literature highlights several limitations. Many application generation platforms rely on one-shot generation, where the application is produced in a single step without explicit requirements validation or user sign-off [7][8]. This approach struggles with complex or ambiguous requirements and often produces incomplete or inconsistent implementations.

In addition, these platforms give limited transparency. Users are not shown how their requirements are interpreted, and there is usually no structured representation of

requirements that can be reviewed, confirmed, and traced back to the generated code [8].

2.3 No-Code and Low-Code Development Platforms

No-code and low-code platforms aim to reduce the need for programming by letting users build applications with visual interfaces. Systems such as Bubble let users create applications without writing traditional code [9]. These platforms are effective for simple use cases and rapid prototyping.

However, the literature identifies several drawbacks. No-code platforms often limit customization and scalability, which makes them unsuitable for complex systems [10]. In many cases, users have limited access to the generated source code, which can lead to vendor lock-in. Furthermore, these platforms do not support formal requirements engineering, requirement confirmation, or systematic validation, which are essential for building reliable software systems [2][8].

2.4 Multi-Agent LLM Systems

Recent research has proposed splitting the cognitive load of a large task across multiple specialized LLM agents instead of relying on a single model. Wooldridge introduces the foundations of multi-agent systems as a research field [12], and more recent work by Xi et al. shows how LLMs can be organized into distinct roles that cooperate to complete complex tasks [14]. In this pattern, one agent might handle planning, another might handle execution, and a third might handle verification. This helps reduce context degradation, keeps each agent's prompt focused, and makes the overall system easier to debug because a failure can be located at a specific agent.

However, most existing multi-agent research focuses on general reasoning tasks rather than end-to-end software generation, and few systems combine multi-agent orchestration with a strict requirements-first workflow and a subprocess-verified deployment pipeline. DeMaestro is designed to fill this gap.

3.0 General Description

DeMaestro is a web-based platform designed to act as a virtual software engineering agency. The system bridges the gap between high-level, unstructured system requirements and practical, executable software implementation. It addresses a fundamental challenge in modern software engineering: turning conceptual ideas and informal documents directly into a functional, runnable full-stack web application without losing the user's original intent.

3.1 System Objectives

The main objective of DeMaestro is to automate the end-to-end creation of full-stack software while enforcing strict requirements-validation rules. Unlike standard code-generation wrappers, DeMaestro runs a multi-stage lifecycle:

1. **Requirement Extraction:** Users upload a requirements document (PDF) or enter a

natural language prompt through the web interface. The system uses its analytical AI engine to extract entities, relationships, and functional requirements into a strict JSON contract.

2. **Ambiguity Resolution:** The system automatically detects missing constraints (for example, unspecified database relations or missing frontend technology preferences) and starts a targeted clarification chat with the user to fill them in.
3. **Explicit Validation:** Before any code is written, the system produces a human-readable Requirements Summary Document. The user must explicitly approve this document, which fulfills the system's requirement-first methodology.
4. **Architectural Blueprinting:** The system turns the approved requirements into a strict architectural blueprint that describes the API routes, database schemas, and component hierarchy.
5. **Code Synthesis and Subprocess Verification:** The specialized execution agents produce the source code for the frontend (React, Tailwind) and backend (Python FastAPI). The centralized orchestrator then runs this code in an isolated OS-level subprocess environment and pipes runtime stack traces directly to the debugging agent so it can patch any syntax or compilation errors.
6. **Live Preview Deployment and Packaging:** Once the code passes internal subprocess verification, the system deploys the application to the cloud using the Vercel Deployments API and provisions an isolated Neon Postgres database for the project. The user can interact with a live serverless instance of the application directly from the frontend, and the source files are also packaged into a downloadable ZIP archive.

3.2 Target User Identification

DeMaestro is designed to serve several user groups within the software development and business ecosystems. By automating the boilerplate and initial architectural setup, the platform provides value to:

- **Software Architects and Senior Developers:** Professionals who design high-level systems and need to quickly prototype or bootstrap the foundational boilerplate of a new application. DeMaestro lets them skip hours of manual directory setup, dependency configuration, and basic CRUD API writing, so they can focus immediately on complex business logic.
- **Entrepreneurs and Small-to-Medium Enterprises (SMEs):** People with concrete software ideas but limited engineering resources. DeMaestro gives them a fast, cost-effective way to turn business requirements into an executable Minimum Viable Product (MVP) that can be handed off to developers for further scaling.
- **Software Engineering Students and Junior Developers:** Students learning full-stack development who need to understand the practical transition from theoretical requirements to executable code. The system works as an educational bridge, showing how abstract constraints map to real file structures, database schemas, and API endpoints.

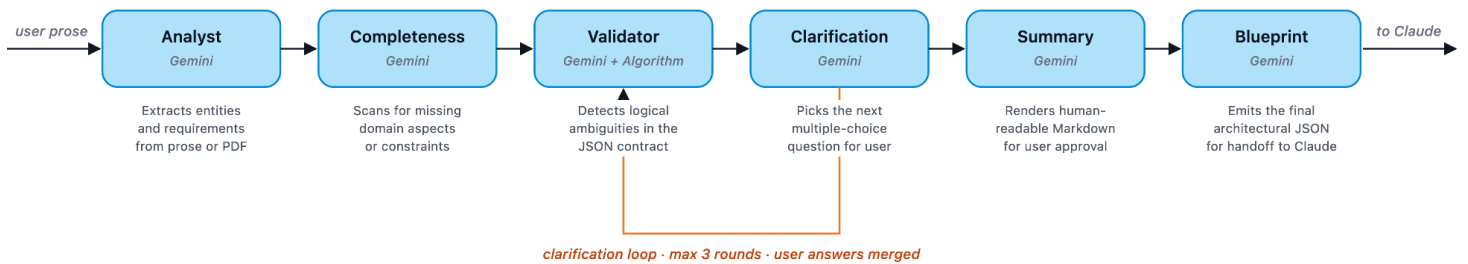
3.3 The Agent Architecture: Centralized Orchestrator Multi-Agent Pattern

To reduce the risk of context window degradation during end-to-end full-stack generation, DeMaestro implements a centralized Orchestrator Multi-Agent Pattern. Instead of asking one large language model to run multiple free-form development

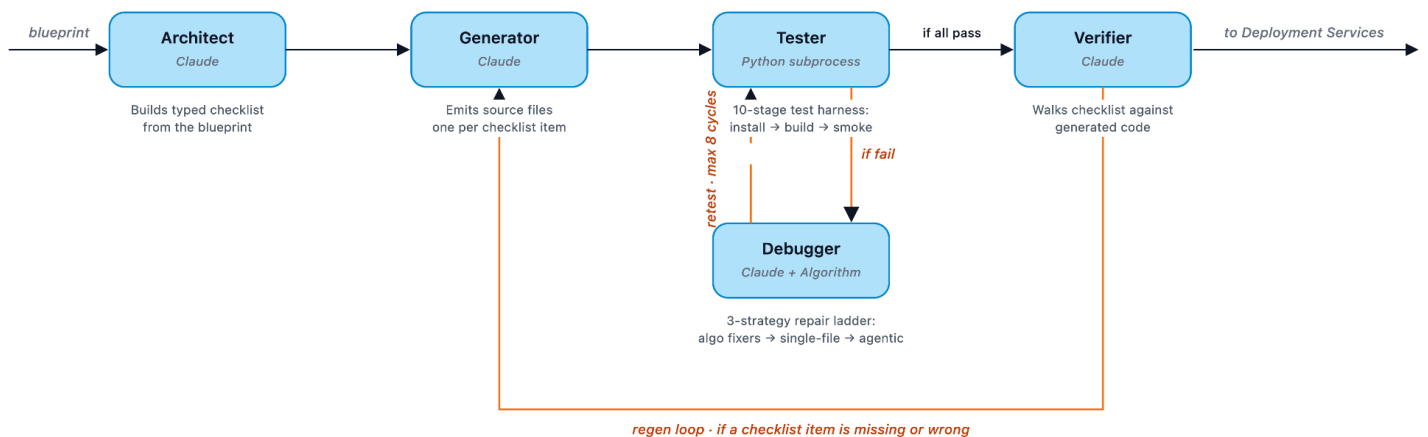
phases, the platform splits the work across a set of specialized components. In total, the system uses roughly a dozen specialized agents, coordinated by two centralized Python orchestrators that act as the deterministic core of the application server.

The ecosystem is divided into two logical groups:

- **Analytical Orchestrators (Gemini-Powered):** Six agents on the Gemini side handle all analytical reasoning. These include the Analyst (which extracts requirements from prose), the Completeness checker, the Validator, the Clarification picker, the Summary writer, and the Blueprint agent that emits the final architectural JSON. Figure 3.1 shows the six Gemini agents and the clarification loop:



- **Execution and Debugging Agents (Claude-Powered):** Five agents on the Claude side handle code production. These include the Architect (which builds a typed checklist from the blueprint), the Generator (which emits source files), the Tester (a ten-stage test harness), the Debugger (a three-tier repair ladder), and the Verifier (which walks the checklist against the generated code). Figure 3.2 shows the five Claude agents, the Test↔Debug loop, and the Verifier regen loop



We use two foundational LLM providers to balance analytical reasoning against raw coding capability:

- **Google Gemini Engine:** Powers the analytical orchestrators and structural extraction agents because of its strong performance on structural schema adherence and long-context comprehension for multi-page PDF parsing [16].
- **Anthropic Claude Engine:** Powers the code execution and debugging agents because of its consistent performance on granular multi-file code synthesis and

real-time error interpretation.

The combination of these roles under centralized orchestration prevents early-stage requirement drift from propagating into the final generated code.

4.0 Development Process

The development of DeMaestro followed an iterative, systematic engineering approach. Because generative AI introduces non-deterministic behavior into software pipelines, traditional waterfall development was not enough. Instead, we adopted an agile, five-phase development lifecycle so we could continuously test, validate, and constrain the AI models throughout the project.

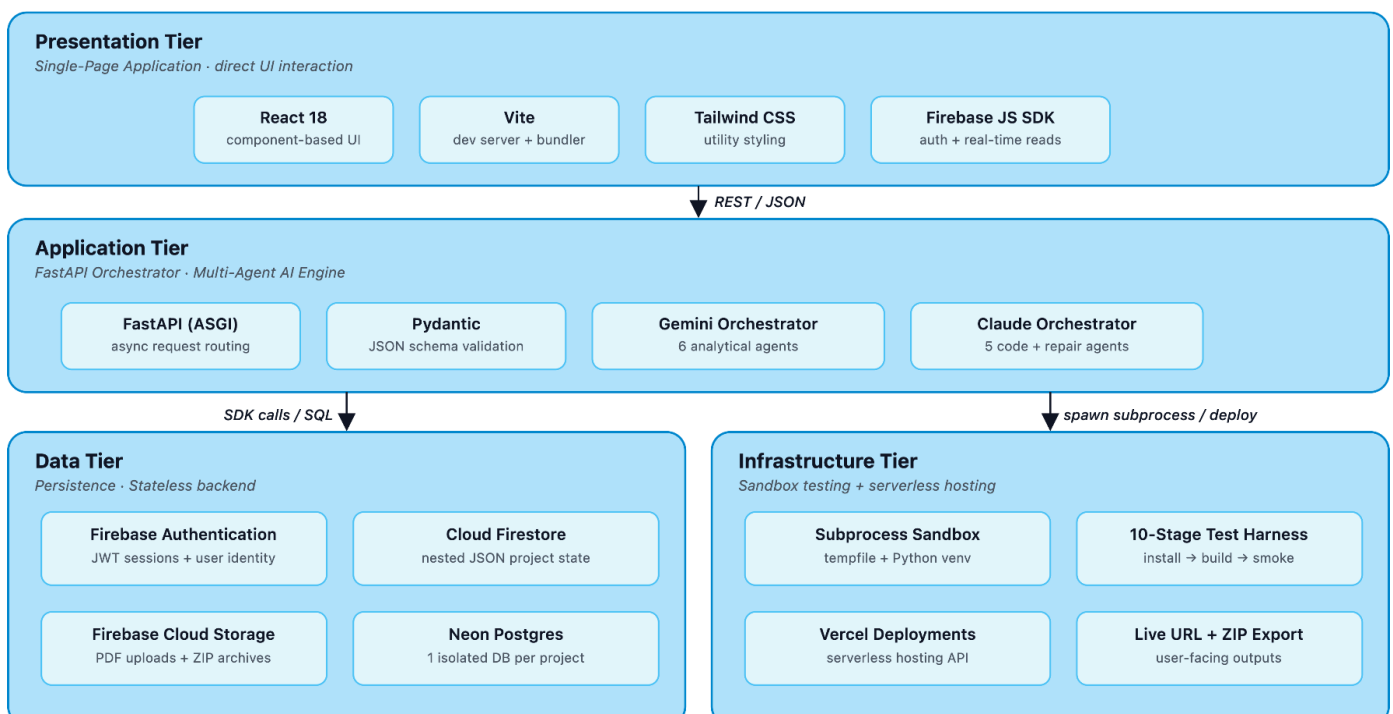
4.1 Phase 1: Research and Planning

We began by researching existing Large Language Models (LLMs) to identify the most suitable engines for full-stack software synthesis. We evaluated Anthropic's Claude and Google's Gemini in detail through prompt testing on raw natural language requirements.

- We discovered that relying on a single model for the entire pipeline led to context degradation and hallucinated file structures.
- We established our core functionality requirements: natural language and PDF parsing, an interactive clarification loop, and an explicit user approval gate.
- We designed our Orchestrator Multi-Agent strategy: assigning the Gemini engine to power the analytical structuring and blueprinting agents, and the Claude engine to power the code generation and debugging agents.
- In parallel, we defined our non-functional requirements (NFRs), including a 60-second limit for requirement submission and a 5-minute maximum threshold for the complete generation and packaging process.

4.2 Phase 2: Architecture Design

After the research phase, we designed a modern, decoupled three-tier architecture that separated user interaction from the AI orchestration logic. Key architectural decisions included:



- **Frontend:** React and Vite for a responsive, state-driven user interface that could handle asynchronous chat interactions and real-time generation updates.
- **Backend:** Python and FastAPI, chosen over Flask because of FastAPI's native asynchronous support (ASGI) and its deep integration with Pydantic for validating JSON schemas produced by the specialized agents.
- **Database and Persistence:** Google Firebase (Authentication, Firestore, and Cloud Storage) so the FastAPI backend could remain stateless and support secure project isolation and horizontal scalability. Neon Postgres was used as a per-project transactional database provisioned at deployment time.
- **Execution and Deployment Infrastructure:** An isolated OS-level subprocess testing environment to safely run and verify AI-generated code locally, plus a serverless Vercel cloud deployment pipeline to host the final application for live user previews [17].

4.3 Phase 3: The Development Phase

The core development was done incrementally, with tasks divided across the stack:

- **Backend Development (FastAPI):**
 - Implemented RESTful API endpoints for file uploads, requirement processing, and project retrieval.
 - Built the integration layers for both the Gemini API and the Claude API.
 - Developed strict Pydantic models to validate the "Hand-off Contract" — the JSON architectural blueprint passed from Gemini to Claude.
 - Built the file-packaging logic to dynamically generate project directory structures and compress them into downloadable ZIP archives.
- **Frontend Development (React):**
 - Created responsive UI components with Tailwind CSS, ensuring a consistent design system.
 - Developed the drag-and-drop PDF upload interface and the interactive chat interface for the clarification loop.
 - Built the Markdown rendering component that safely displays the Requirements Summary Document for user approval.
 - Added real-time feedback: loading skeletons, toast notifications, and dynamic progress indicators during the generation window.
- **Subprocess Environment and Deployment Infrastructure:**
 - Wrote Python orchestration scripts that use the `tempfile` module to provision isolated, per-project directory structures (`tempfile.mkdtemp`) on the host server.
 - Built runtime execution safety mechanisms using Python's native `subprocess` module to spin up isolated OS-level Python Virtual Environments (`python -m venv`) for per-project verification.
 - Built a serverless cloud deployment pipeline using the Vercel Deployments API to package and host both the React frontend and the FastAPI backend without local port overhead.
 - Wrote error-handling code that collects execution stack traces from the local subprocess environment and pipes them directly back to the debugger agent for recursive repair.

4.4 Phase 4: Integration and Testing

Once the individual layers were working, we integrated all system components and made sure the asynchronous communication between the React client, the FastAPI orchestrator, Firebase, the multi-agent ecosystem, and the OS-level subprocess environment worked correctly.

- **Unit Testing:** We tested the Pydantic validation layers by intentionally sending malformed AI responses to make sure the backend caught the exceptions and forced the orchestrator to retry, instead of corrupting the system state.
- **Integration Testing:** We ran end-to-end workflows using various complex PDF requirement documents and natural language prompts, and measured how well the system mapped dependencies across the generated React frontend and FastAPI backend inside the virtualized testing environment.
- **Client Interface Feedback:** We exposed our interactive prototypes to peer software engineering students. Their feedback showed that open-ended AI clarification questions caused user fatigue, which directly led to our decision to build a multiple-choice clarification UI.

4.5 Phase 5: Refinement and Optimization

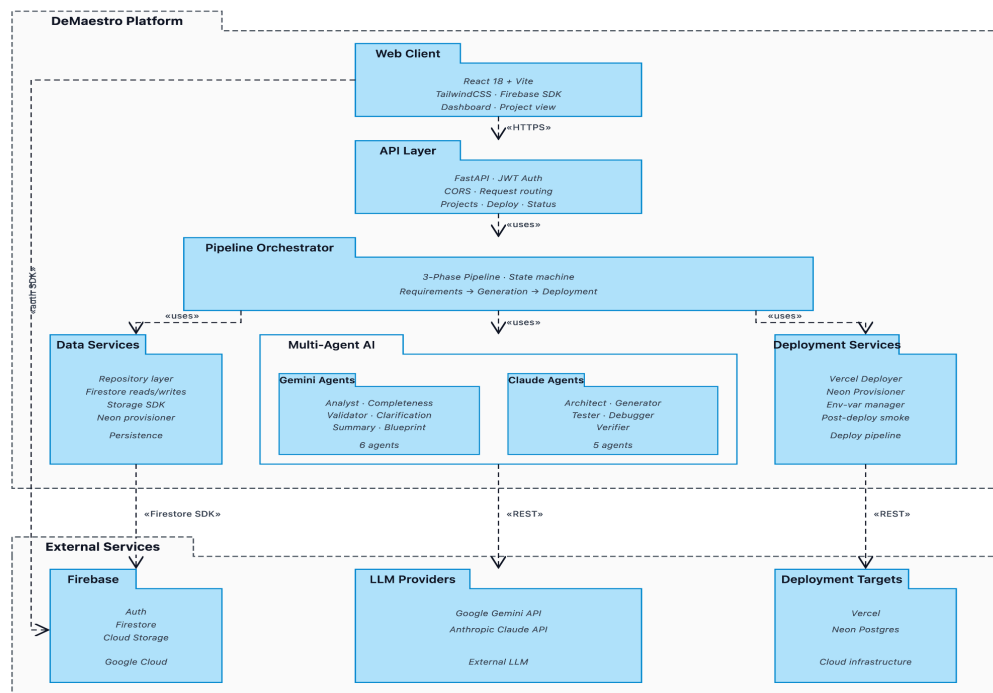
Based on the integration testing results and user feedback, we iteratively refined the system:

- **Algorithmic Refinement:** To solve the user fatigue issue from Phase 4, we changed our requirements-gathering algorithm. We constrained the AI to offer guided, multiple-choice options (for example, a curated menu of supported databases) instead of open-ended questions, which improved usability.
- **Prompt Engineering:** We optimized the system prompts sent to Gemini and Claude, reducing token usage and improving the precision of the generated code.
- **Error Handling:** We improved the frontend error boundaries so they can handle API timeouts and show user-friendly troubleshooting messages.

5.0 System Architecture

The DeMaestro architecture is designed to produce deterministic, verifiable outcomes from non-deterministic generative AI models. To do this, the system separates the user interface completely from the AI processing logic and uses a strict five-layer pipeline. Communication between the layers is unidirectional and managed by a central orchestration server.

5.1 System Package Diagram



The Package Diagram (Figure 5.3) groups the codebase into five architectural layers inside the DeMaestro Platform container, plus three external service groups. The Activity Diagram (Figure 5.2) shows how a single project flows through these layers over time. The following sub-sections describe each layer's responsibilities, technology choices, and design decisions.

5.1.1 The Frontend Layer (Web Client)

The frontend of DeMaestro is built as a Single-Page Application (SPA) using React 18, Vite, and Tailwind CSS. The application follows a component-based architecture and encapsulates complex UI states into reusable modules.

- **State Management:** Because AI generation involves long-running asynchronous tasks, the frontend uses React Hooks together with a manual polling pattern (built on `useEffect` and a `pollNonce` counter) to manage loading states, so the UI does not freeze while waiting for backend responses.
- **Component Design:** The interface includes a drag-and-drop file upload zone, a real-time clarification chat interface, and a Markdown renderer that safely displays the AI-generated Requirements Summary Document.
- **Communication:** The frontend communicates with the FastAPI backend through RESTful API calls for pipeline operations, and directly with Firebase (via the Firebase JS SDK) for authentication and real-time project state updates. It never communicates directly with the LLM providers — all Gemini and Claude calls go through the backend.

5.1.2 The Backend Orchestration Layer

The backend is implemented with Python 3.11 and the FastAPI framework. It acts as the central brain of the platform, routing requests, running the generation pipeline, and enforcing structural validation.

- **Asynchronous Operations:** We chose FastAPI for its native ASGI (Asynchronous Server Gateway Interface) support, which lets the server handle concurrent AI processing requests without blocking other users.
- **JSON Schema Validation:** Generative AI is prone to structural hallucination. The backend uses Pydantic to define strict data models. Every response from the AI agents must match these Pydantic schemas. If validation fails, the backend catches the exception and automatically forces the AI to retry, so no malformed data can move through the pipeline.
- **Workspace Management:** The backend generates isolated directory structures for each project, fills them with the generated code, and compresses them into ZIP archives for the user to download.

5.1.3 The Data and Persistence Layer

DeMaestro uses Google Firebase together with Neon Postgres as its persistence layer, which lets the FastAPI backend stay stateless.

- **Firebase Authentication:** Manages secure user registration and issues JWT (JSON Web Tokens) to authenticate API requests, which ensures cross-project data isolation.
- **Cloud Firestore:** A NoSQL document database used to store the nested JSON structures produced by the AI, including raw user inputs, structured requirement contracts, and the final architectural blueprints.
- **Cloud Storage:** Acts as the binary blob repository. It securely stores the original PDF documents uploaded by users and hosts the final compiled ZIP archives containing the generated source code.
- **Neon Postgres (per-project):** For every deployed project, DeMaestro provisions an isolated Postgres database on Neon, initializes its schema from the blueprint, and injects the connection string into Vercel's environment variables. This gives each generated application its own transactional data store with cross-tenant isolation at the database level.

5.1.4 The AI Agent Engine Layer

To prevent context degradation during complex software generation, DeMaestro replaces single-model generation with a Centralized Orchestrator Multi-Agent Pattern.

- **Analytical Orchestrators (Gemini-Powered):** The Gemini API powers the central planning orchestrators. These components handle all analytical reasoning: parsing the initial prompt, running the clarification loop, and producing the rigid architectural JSON blueprint.
- **Execution and Debugging Agents (Claude-Powered):** The Claude Agent SDK powers the execution ecosystem — a specialization ladder that includes the Architect (checklist builder), the Generator (file emitter), the Tester (10-stage test

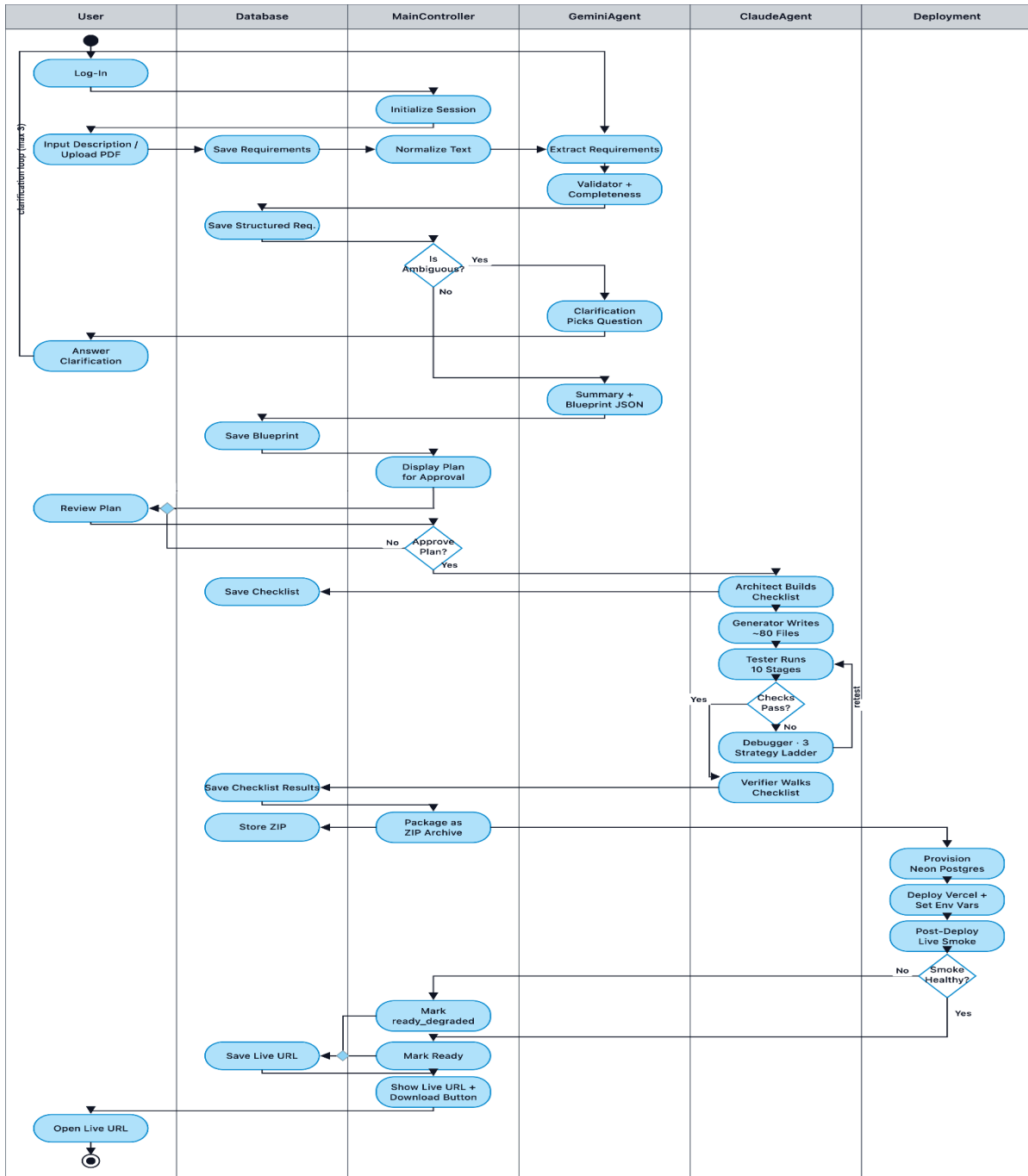
harness), the Debugger (3-strategy repair ladder), and the Verifier (checklist walker). The Debugger escalates from 60 algorithmic Python fixers, to targeted single-file LLM regeneration, to an agentic project-wide fix, capped at 8 test-debug cycles per project.

5.1.5 The Subprocess Execution and Cloud Deployment Layer

Because AI-generated source code cannot be trusted blindly, DeMaestro uses a dedicated OS-level execution layer to test the code and a serverless cloud pipeline to host it.

- **Sandbox Testing (Subprocess):** For every generation, the backend creates a fresh isolated workspace with `tempfile.mkdtemp()`, spins up a `python -m venv`, installs dependencies in bounded worker threads, and runs a 10-stage test harness (install, lint, typecheck, boot, build, smoke, reachability, contract, FE↔BE match, browser smoke) — all through Python subprocesses. Wall-clock timeouts protect the host from runaway processes.
- **Cloud Deployment (Vercel + Neon):** Once the sandbox passes, the Deployment Services push the whole repository to Vercel through its Deployments API, provision an isolated Neon Postgres database, wire up the environment variables, and run a live post-deploy smoke test against the deployed URL. A `ready_degraded` fallback is applied if the live smoke test fails but the application is still reachable.

5.2 System Flow



The operational workflow of DeMaestro is designed as a phased pipeline. The system strictly halts at the approval gate, which prevents "one-shot" AI hallucinations and ensures complete user traceability.

1. **Authentication and Input:** The user logs in with Firebase Authentication and starts a project by uploading a PDF requirements document (up to 10 MB) or entering a natural language description.
2. **Structuring Phase:** The FastAPI backend sends the text to the Gemini agent, which maps the unstructured data into a defined JSON requirements contract.
3. **The Clarification Loop:** The system analyzes the JSON contract. If variables are

missing (for example, unspecified database frameworks), the backend enters a clarification state. The frontend shows multiple-choice queries to the user. The loop repeats until the contract is logically complete.

4. **The User Approval Gate:** Once clarified, Gemini translates the JSON contract into a human-readable Markdown document (the Requirements Summary). Execution halts here. The user must read and explicitly approve this document to continue.
5. **Blueprint Generation:** After user approval, Gemini produces the final architectural blueprint, mapping out API routes, database models, and frontend UI components.
6. **Code Synthesis and Subprocess Verification:** The architectural blueprint is given to the specialized execution agents, which produce the frontend and backend source code files. The centralized orchestrator provides a transient OS-level virtual environment, runs the code with Python subprocesses, and pipes any resulting stack traces back to the debugging agent for iterative syntax patching.
7. **Live Preview Deployment:** Once the generated code passes the sandbox test suite, the Deployment Services push the project to Vercel via its Deployments API and provision a per-project isolated Neon Postgres database. A post-deploy smoke test walks the live endpoints; if healthy, the project is marked ready and the live URL is stored in Firestore.
8. **Packaging and Export:** In parallel with the cloud deployment, the backend zips the workspace, uploads it to Firebase Cloud Storage with a signed URL, and stores the URL in Firestore. The dashboard shows both the Live URL and the Download ZIP buttons.

6.0 Challenges and Obstacles

Throughout the development of DeMaestro, we ran into several analytical and technical challenges. Building an autonomous software generator requires overcoming the natural unpredictability of Large Language Models. This section describes the main obstacles we faced, the engineering solutions we used, and the lessons we learned.

6.1 Enforcing Output Determinism from Generative AI

One of the most significant challenges in building an automated AI pipeline is structural hallucination. Generative models naturally produce free-form text and often add conversational filler (for example, "Here is your code...") or drift away from the requested JSON structure. In an automated pipeline, a single missing comma or hallucinated JSON key will crash the backend parsing logic.

- **The Solution:** We removed the AI's free-form chat capabilities at the backend orchestration level. We defined strict Pydantic models in our FastAPI server to describe the exact shape of the "Hand-off Contract" (the blueprint). Every prompt sent to Gemini explicitly demands this JSON structure. If the AI deviates, Pydantic raises a validation exception, and our backend catches it and automatically triggers a regeneration attempt. This gave us structural determinism before the data even reached the code generation phase.

6.2 Host-Safe Execution via Subprocess Virtualization

In Phase A, verifying full-stack integration at runtime was identified as a critical requirement. However, executing unverified AI-generated source code on the main host

server creates serious security and stability risks, including resource starvation, system file corruption, and infinite loops that block the async core of the FastAPI application server.

- **The Solution:** We built an OS-level Virtual Environment Sandbox that runs on isolated, transient directory trees. When Claude produces code, the backend isolates the project with `tempfile.mkdtemp()`. It creates and activates a fresh Python virtual environment and runs the compilation check inside bounded, non-blocking asynchronous subprocesses through Python's `subprocess` module. Wall-clock timeouts on the worker threads cleanly kill runaway processes, so we protect the host's hardware limits while isolating dependencies entirely inside the transient project directory.

6.3 The Debugger Strategy Ladder

The subprocess sandbox from Section 6.2 gives us a reliable failure signal, but it does not tell us *how* to fix a broken project. A single LLM call to "fix these errors" usually produces one of three outcomes: it patches the reported symptom without fixing the root cause, it introduces cross-file regressions while patching the symptom, or it rewrites the whole project and undoes work that was already correct. Just calling the LLM in a loop wastes tokens, drifts away from the original blueprint, and often ends up in a worse state than the starting point.

- **The Solution:** We built a three-tier escalation ladder that tries the cheapest, most targeted repair first and escalates only when it fails.
 - **Strategy 1 — Algorithmic Fixers (no LLM).** A library of 60 hand-written Python patches recognizes and repairs recurring failure signatures that we observed in early testing: missing imports, mismatched Pydantic field types, unregistered FastAPI routes, forgotten CORS headers, wrong Python venv paths, unused variables that trip lint, and dozens more. Each fixer runs in milliseconds and touches only the offending line — no LLM tokens are spent.
 - **Strategy 2 — Targeted Regeneration (single-file LLM).** If Strategy 1 leaves errors behind, the Debugger reads the stack trace, identifies the failing file, packages just that file plus its immediate imports, and asks Claude to regenerate the single file with the error message attached as context. The rest of the project stays untouched.
 - **Strategy 3 — Agentic Holistic Fix (whole-project LLM).** Only when Strategy 2 also fails does the Debugger fall back to the most expensive option: sending the entire project plus the full test output to Claude in an agentic session, and letting it decide which files to modify. This strategy is capped at 5 invocations per project to prevent runaway cost.

Between each strategy the Tester re-runs, and a project snapshot is captured before every fix so a bad patch can be rolled back instead of committed. The whole Test↔Debug loop is capped at 8 cycles per project — if all three strategies exhaust their budget, the project is marked `ready_degraded` and shipped with the last-known-good snapshot instead of looping forever. This tiered escalation is the main reason DeMaestro's projects reach deployment at a higher rate than a naïve LLM-loop baseline: cheap fixes handle the common case, and the expensive fix is used only for the rare hard case where it is really needed.

6.4 Serverless Cloud Deployment via Vercel

Showing the user a live preview of a dynamically generated full-stack web application originally required complex local networking and proxy management. Dynamically allocating open host network ports and writing transient custom Nginx reverse-proxy configuration files creates serious security vulnerabilities and single-point-of-failure risks on the orchestration server.

- **The Solution:** We removed the local networking problem entirely by building a Serverless Cloud Deployment Pipeline that uses the Vercel Deployments API. When an application passes its internal subprocess verification, a dedicated deployment controller packages the frontend and backend files and provisions an isolated Neon Postgres database for that project. It then pushes the generated assets to Vercel's cloud, injects the Neon connection string as an environment variable, and starts a remote build. Vercel then allocates cloud-managed endpoints and gives us an isolated, live deployment URL. This URL is stored in Firestore and returned to the DeMaestro user interface, so the user can view and interact with a live instance of the generated MVP without putting any routing or container-exposure burden on our infrastructure.

6.5 Lessons Learned

The project achieved its main objective of bridging the semantic gap. The development process also gave us several important insights for future iterations:

1. **The Necessity of Multi-Agent Orchestration:** Our biggest takeaway is that unconstrained single-model LLMs cannot reliably generate complex software systems because of context window degradation. Splitting the cognitive load across our specialized agents — Gemini for architectural reasoning, Claude for localized syntax and repair — was the only way to reach reliable full-stack integration.
2. **API Latency and Cost Constraints:** As described in Section 6.3, our tiered Debugger ladder controls cost by trying cheap algorithmic fixes before expensive LLM calls. Even so, the projects that reach Strategy 3 (agentic holistic fix) consume significant tokens, and the full Test↔Debug loop can add several minutes to the generation time. In a future iteration, we would add a caching layer for recurring code blocks (like standard Firebase authentication setups) to reduce token consumption and speed up generation.
3. **Database Migration Limitations:** Currently, DeMaestro provisions a fresh database schema for every generation. If a user wants to iteratively update an existing project, the system struggles to produce safe database migration scripts (for example, Alembic for Python). Prioritizing stateful, iterative generation — where each subsequent run diffs against the previous blueprint and produces migration steps instead of a fresh schema — is a key goal for future development.

7.0 Project Metrics

In Phase A, we defined a set of Non-Functional Requirements (NFRs) to evaluate the effectiveness and reliability of DeMaestro. Our empirical testing focused on two critical

aspects: System Performance (latency) and Code Reliability. For readability, we restate the relevant NFRs inline where they are evaluated.

7.1 System Performance and Generation Time

System performance was identified as a critical metric for the platform's practical usability.

- Requirement Gathering (NFR5 — the clarification loop must finish in under 60 seconds). The system was required to let users submit requirements and complete the clarification loop in under 60 seconds. Because the React client manages state locally and only queries the Gemini API for specific ambiguities, we met this metric.
- Full-Stack Synthesis Time (NFR6 — full generation and packaging must complete within 5 minutes). During testing, simple applications (for example, basic CRUD dashboards) completed the pipeline in 2 to 3 minutes. Highly complex applications sometimes approached the 4.5-minute mark because the debugging agent needed multiple recursive loops inside the isolated subprocess environment. The system stayed within the 5-minute requirement and provided real-time UI loading states to keep the user engaged during processing.

7.2 Code Reliability and Validity Rate

The core metric for DeMaestro's success was the Code Validity Rate — the percentage of generated projects that successfully compiled and booted without syntax errors or immediate crash loops (NFR2 — every generated project must compile; NFR3 — every generated project must boot without a runtime crash).

- Pre-Virtualization vs. Post-Virtualization: During early Phase B development (before the OS-level subprocess verification environment was integrated), the validity rate hovered around 40%. Mismatched API routes between the React frontend and FastAPI backend, and unverified dependency imports, frequently caused silent runtime failures.
- Final Evaluation: After we implemented the Orchestrator Multi-Agent Pattern and the automated subprocess testing loop, the validity rate improved significantly. By piping local execution stack traces directly back to the debugging agent, the system recursively patches the syntax before the final Vercel deployment and ZIP packaging. As a result, projects synthesized by DeMaestro now execute with reliable interface connections and meet our defined reliability constraints (NFR2 and NFR3).

7.3 Usability and Interface Effectiveness

To make sure the system met the needs of our target audience, we ran usability sessions with peer software engineering students. Early feedback showed that open-ended chat interactions with the AI caused friction. By updating the frontend to support guided, multiple-choice parameter selection during the clarification phase, user confidence and completion rates increased, which validated our requirement-first methodology.

References

- [1] Brown, T. et al., "Language Models are Few-Shot Learners," NeurIPS, 2020.
- [2] Sommerville, I., *Software Engineering*, 10th Edition, Pearson, 2015.
- [3] GitHub Copilot, product page.
- [4] Amazon CodeWhisperer, product page.
- [5] Vaithilingam, P. et al., "Expectations, Outcomes, and Challenges of Using AI-Based Code Generation Tools," CHI, 2022.
- [6] Replit, product website.
- [7] Chen, M. et al., "Evaluating Large Language Models Trained on Code," arXiv:2107.03374, 2021.
- [8] Nuseibeh, B., and Easterbrook, S., "Requirements Engineering: A Roadmap," 2000.
- [9] Bubble, product website.
- [10] Poppendieck, M., and Poppendieck, T., *Lean Software Development*, Addison-Wesley, 2003.
- [11] Kulkarni, C. et al., "Semantic Gaps in AI-Assisted Programming," ACM TOSEM, 2021.
- [12] Wooldridge, M., *An Introduction to Multi-Agent Systems*, 2nd Ed., Wiley, 2009.
- [13] Stone, P. et al., "Artificial Intelligence and Life in 2030," Stanford, 2016.
- [14] Xi, Z. et al., "The Rise and Potential of Large Language Model Based Agents," arXiv:2309.07864, 2023.
- [15] Pressman, R. S., and Maxim, B. R., *Software Engineering: A Practitioner's Approach*, 8th Ed., McGraw-Hill, 2014.
- [16] Google, "Gemini: A Family of Highly Capable Multimodal Models," Google AI Blog, 2023.
- [17] Humble, J., and Farley, D., *Continuous Delivery*, Addison-Wesley, 2010.

Appendices

Appendix A – User Guide

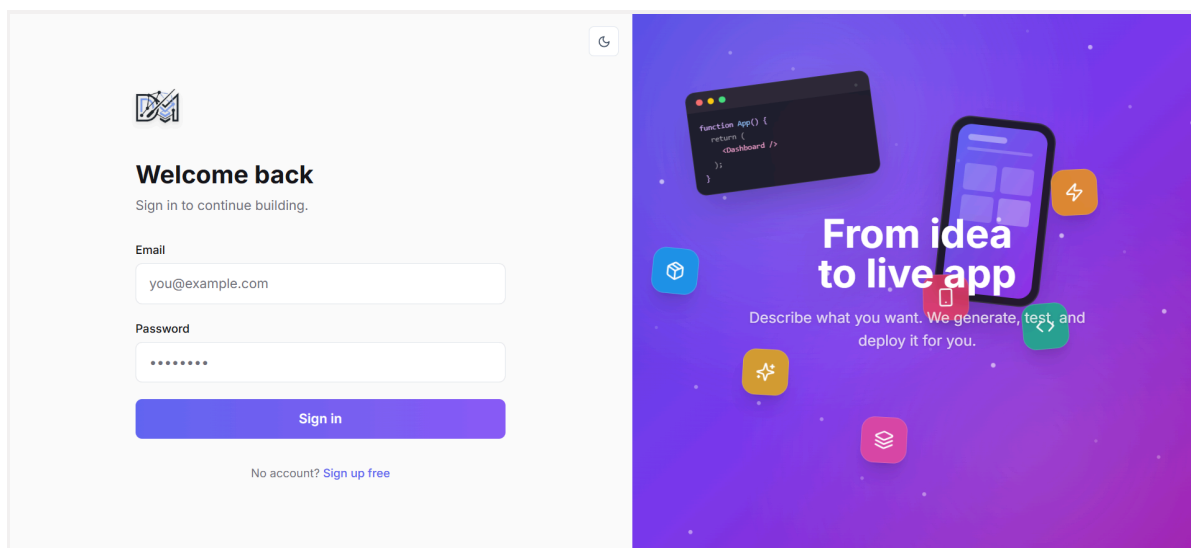
A.1 Introduction

DeMaestro is an autonomous, web-based platform designed to bridge the gap between unstructured software ideas and executable full-stack implementations. This tool translates natural language prompts or PDF requirements documents into functional React and FastAPI source code. By automating the boilerplate generation and enforcing a strict requirement-validation loop, DeMaestro allows software engineers, students, and entrepreneurs to move from concept to a live deployment in minutes.

This user guide provides step-by-step instructions for operating the DeMaestro platform. It covers the complete "happy path" workflow, from creating an account and submitting requirements to approving the architectural summary and downloading the final generated source code.

A.2 Operating Instructions

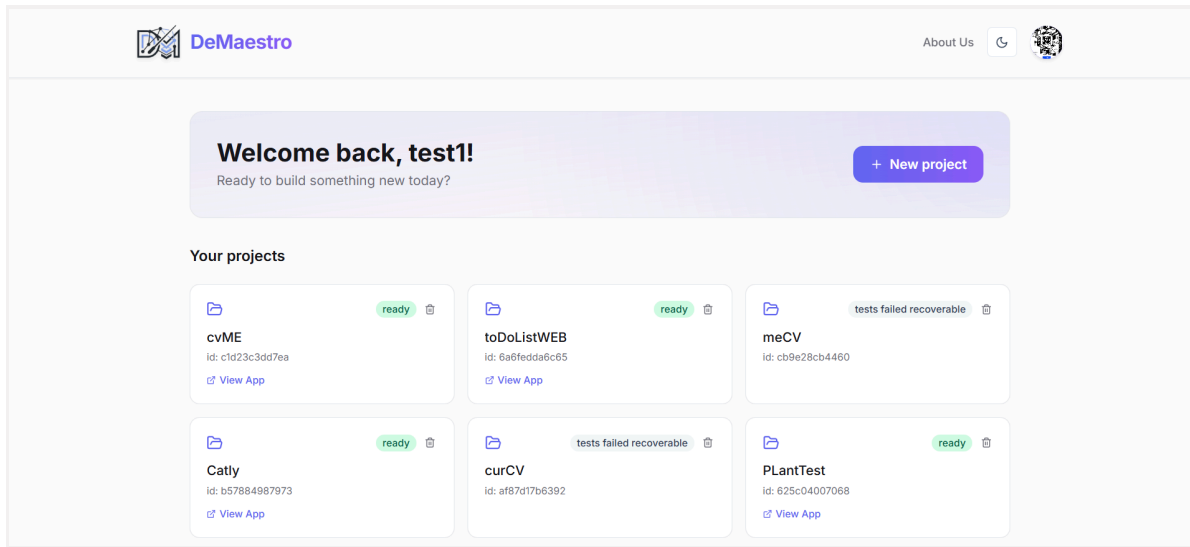
A.2.1 Sign-Up and Authentication Page



To utilize DeMaestro's cloud generation features and save your project history, you must first create an authenticated account.

1. Navigate to the DeMaestro web application.
2. Click on the **Signup** option in the top navigation bar.
3. Fill in the registration form with your credentials (Email and Password).
4. Click the **Sign Up** button to complete registration.
5. If you already have an account, select the **Login** tab, enter your registered credentials, and click **Login**. Upon successful authentication, you will be securely redirected to the main dashboard.

A.2.2 The Main Dashboard



After logging in, you will land on your personal workspace dashboard. This screen displays a grid of your previously generated projects.

- **To view a past project:** Click the **View** button on any project card to access its architectural blueprint, original requirements, and download links.
- **To start a new generation:** Click the prominent **+ New Project** button located at the top of the workspace.

A.2.3 The Requirements Input Page

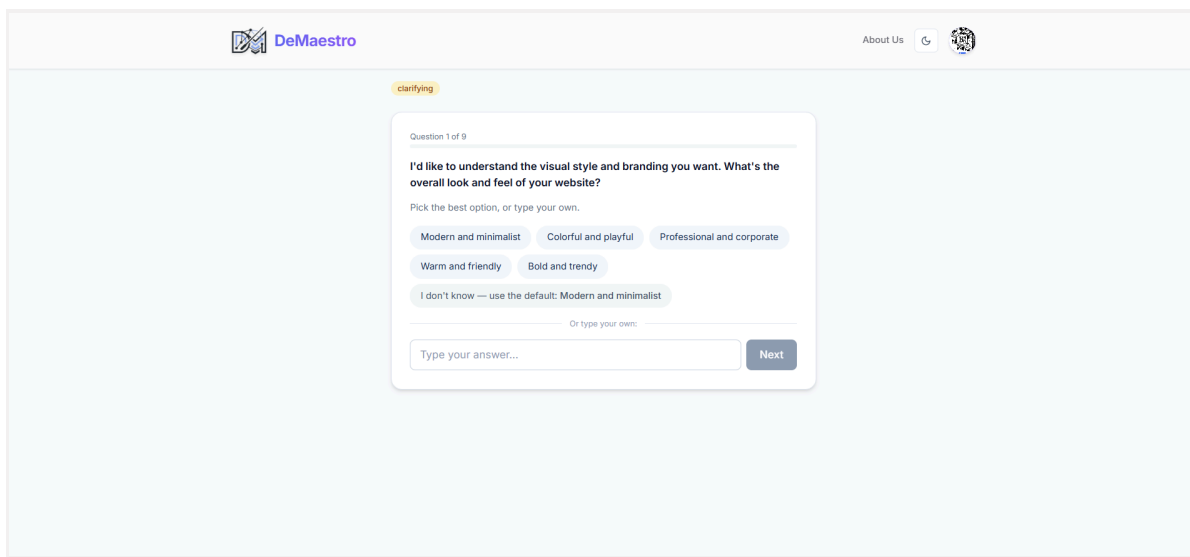
Upon

initiating a new project, you will be presented with the input interface where you define your software idea.

1. Upload Options:

- **Document Upload:** Drag and drop your requirements PDF (up to 10MB) into the designated upload area, or click to browse your local files.
 - **Text Prompt:** Alternatively, use the text area to type a natural language description of your desired web application.
2. Once your file is attached or your text is entered, click the **Analyze Requirements** button. The system will transition to a loading state while the AI agent extracts the core entities and logic.

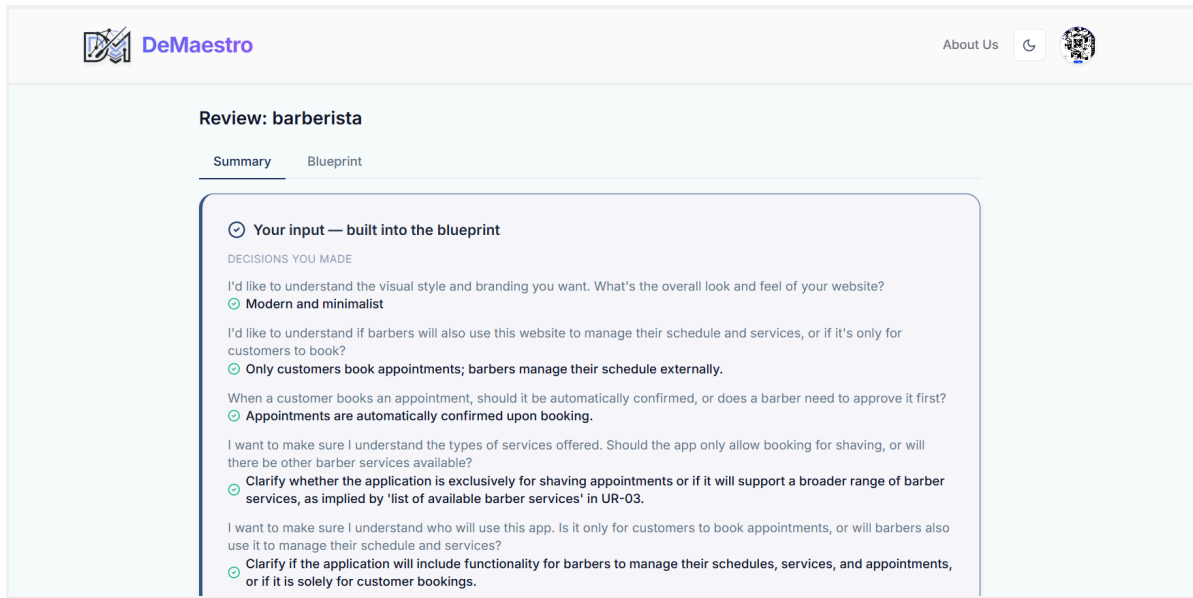
A.2.4 The Clarification Chat Loop



DeMaestro does not blindly guess missing information. If your uploaded requirements lack specific technical constraints (such as a preferred database or missing user roles), the system will halt and prompt you for clarification.

1. The interface will display a chat-like window with a specific question generated by the AI (e.g., *"Which database framework would you prefer for the user authentication system?"*).
2. Below the question, you will see **Multiple-Choice Option Buttons** containing supported technology stacks or logical pathways.
3. Click the option that best fits your needs.
4. If further ambiguities exist, the system will ask follow-up questions. Once all parameters are resolved, the system will automatically advance to the Approval Gate.

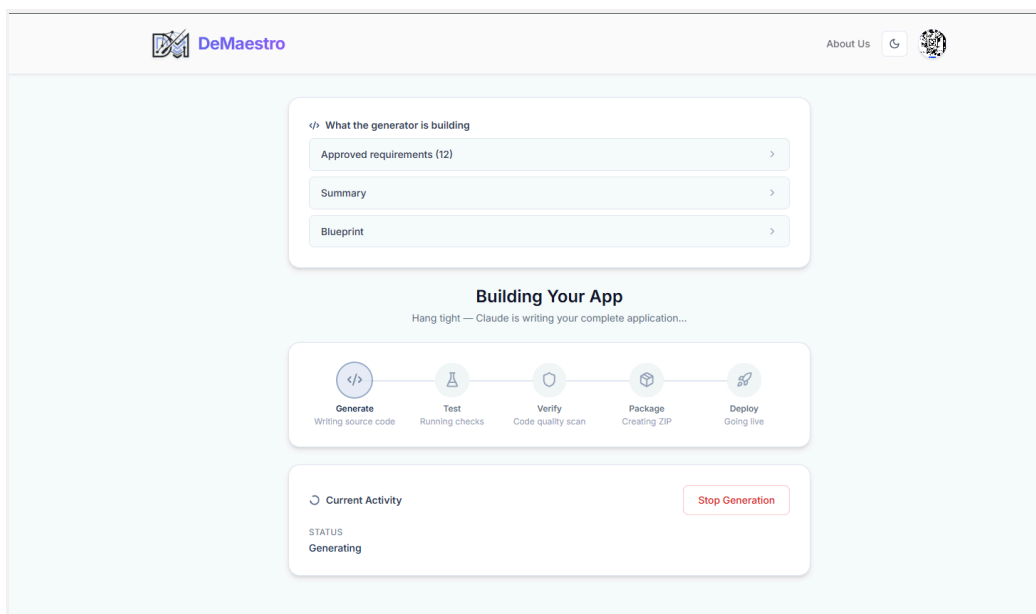
A.2.5 The User Approval Gate



Before any code is generated, DeMaestro enforces a strict validation step to ensure its understanding aligns with your intent.

1. The screen will display a formatted **Requirements Summary Document** (rendered in Markdown).
2. Review this document carefully. It outlines the proposed frontend components, backend API routes, and the database schema the system intends to build.
3. If the document is accurate, click the **Approve and Generate** button.
4. *Note: Once approved, the architectural blueprint is locked, and the code synthesis phase begins.*

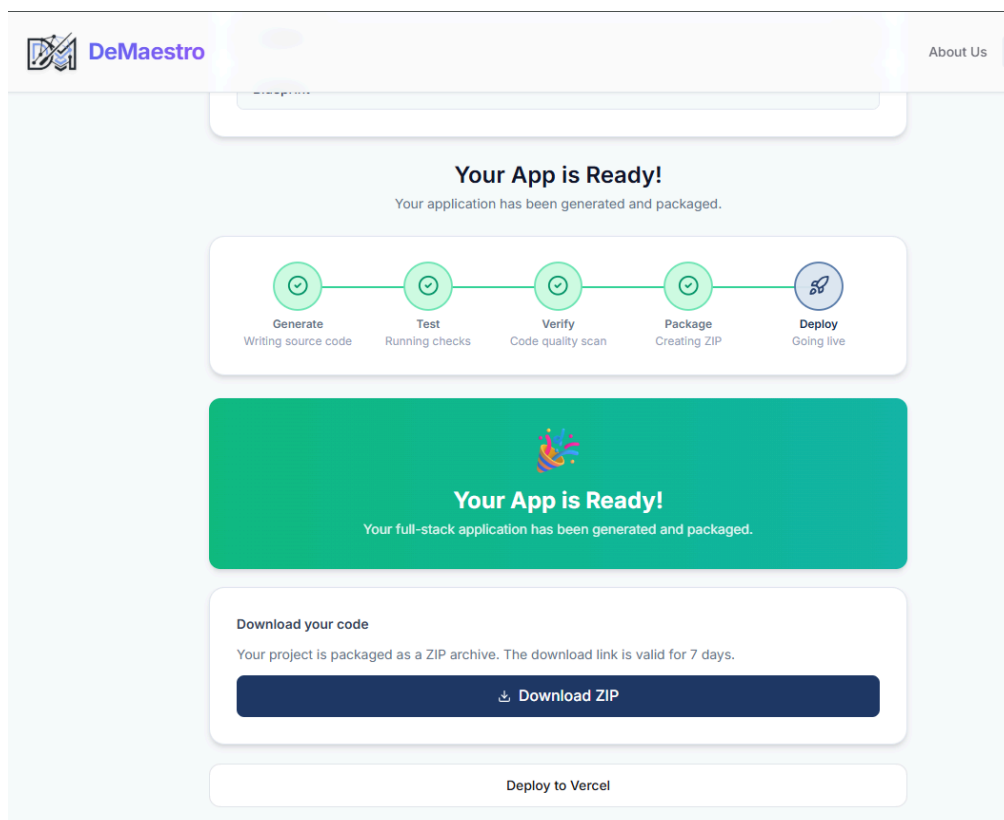
A.2.6 Generation and Sandboxing Status



Because full-stack synthesis, environment virtualization, and automated verification loops require heavy computation, you will see a progress screen.

1. A loading animation will appear with the status text: *"Synthesizing Code and Initializing Virtual Environment..."*
2. A progress indicator will display the specific active stage (e.g., *Generating Frontend Components, Synthesizing Backend Routers, Running Automated Integration Tests*).
3. **Important:** Do not close the browser window during this phase. This process typically takes between 2 to 5 minutes depending on the architectural complexity of the system being generated and the number of debug iterations required by the centralized orchestrator.

A.2.7 Live Preview and Download Page



Once the generation and cloud-managed testing phases complete successfully, you will be automatically redirected to the final Export Page.

1. **Live Preview:** In the center of the interface, an embedded iframe renders the application. Rather than a localized sandbox, this area embeds a secure, live **Vercel deployment URL** generated uniquely for your project. You can fully test, click through, and validate the system UI directly in this window.
2. **Export Options:** Below the preview, you have two primary actions:

- **Download Blueprint:** Downloads the structural JSON file defining the architecture.
- **Download Source Code (ZIP):** Downloads the complete, compressed workspace containing all React, FastAPI, and configuration files, ready to be run on your local machine.

A.3 Navigation and Workspace Management

A.3.1 The Navigation Bar

The top navigation bar remains persistent across all authenticated screens, providing quick access to essential areas:

- **Dashboard:** Returns you to the grid view of your saved projects.
- **New Project:** Instantly aborts the current view and opens a fresh upload screen.
- **Logout:** Securely terminates your active session.

A.3.2 Real-Time Notifications

Throughout your usage of DeMaestro, the system utilizes "toast" notifications (small pop-ups in the corner of the screen) to provide immediate feedback. These will alert you to successful file uploads, invalid file formats, and network timeouts, ensuring you are always aware of the system's operational status.

Appendix B – Maintenance Guide

B.1 Introduction

This Maintenance Guide provides the necessary technical documentation for future DevOps engineers or system administrators responsible for maintaining the DeMaestro platform's core orchestration layer. Because DeMaestro functions as an autonomous generation utility, its infrastructure relies heavily on container orchestration, dynamic port allocation, and multi-agent API management. This guide outlines the specific hardware prerequisites, environment configurations, and step-by-step instructions required to run and maintain the DeMaestro orchestration server and its sandbox environment.

B.2 System Requirements

B.2.1 Hardware Requirements

Because DeMaestro utilizes localized OS-level virtualization to execute and verify code, the host server must maintain sufficient memory allocation to prevent sub-process timeouts during active generation.

- **Processor:** Multi-core CPU (Intel i5 / AMD Ryzen 5 or equivalent or higher).
- **Memory:** Minimum 8 GB RAM (16 GB recommended to handle multiple concurrent multi-agent processing pipelines).
- **Storage:** 5 GB of free disk space (required primarily for virtual environment caching, file asset structuring, and python library requirements).

B.2.2 Software Requirements

The backend orchestration server runs directly on the host machine OS environment. The following dependencies must be installed natively prior to initialization:

- **Python:** Version 3.11 or higher (strictly required for FastAPI core and Pydantic schema validation layers).
- **Node.js & npm:** Version 18.x or higher (required on the host system to run local build/verification checks).
- **Package Manager:** `pip` (Python package manager).

B.3 Configuration and API Credentials

The DeMaestro orchestration backend relies on several external cloud services. The system administrator must provision the following credentials:

1. **Google Firebase:** A Firebase project must be active with Authentication, Firestore, and Cloud Storage enabled. Generate a Firebase Admin SDK private key (JSON file) from the project settings.
2. **Google Gemini API Key:** Required for the Requirements Analyst and Blueprint Architect agents.
3. **Anthropic API Key:** Required for the Claude Code Engineer agent.

The `.env` file in the root directory of the backend must adhere to the following configuration structure:

```
GEMINI_API_KEY=your_gemini_key_here
```

```
ANTHROPIC_API_KEY=your_claude_key_here
```

B.4 Orchestration Backend Setup (FastAPI)

The backend orchestration server must be executed within an isolated Python virtual environment to prevent dependency conflicts with the host system.

1. Navigate to the backend directory on your server.

Create and activate a Python virtual environment:

```
python3 -m venv venv
```

2. `source venv/bin/activate`
3. Install the required Python packages (including FastAPI, Uvicorn, Pydantic, and your LLM engine clients):
`pip install -r requirements.txt`
4. Place the `firebase-admin.json` service account file strictly inside the `config/` directory.
5. Start the FastAPI server using Uvicorn:
`uvicorn main:app --reload`

The orchestration server is now active. System administrators can view the automatic OpenAPI documentation and test routes at <http://localhost:8000/docs>.

B.5 Sub-System Maintenance Guidelines

B.5.1 Modifying the AI Architectural Blueprint

If future maintainers wish to expand the capabilities of DeMaestro (e.g., adding a generation layer for automated UI testing frameworks), the "Hand-off Contract" must be updated. You **must** update the Pydantic schemas located in the `models.py` file. If the Pydantic schemas do not perfectly map to the prompt instructions given to Gemini, the backend validation pipeline will reject the AI's output and trigger an infinite regeneration loop.

B.5.2 Troubleshooting the Serverless Live Preview

The Live Preview deployment relies on the Vercel REST API for serverless cloud hosting. If the generated applications fail to render in the frontend iframe, administrators should first verify that the host server has outbound network access to Vercel's API endpoints. Additionally, check the Python orchestration logs for local subprocess compilation errors that may have prevented the deployment phase from initiating. If the source code was successfully pushed but the iframe fails to load, administrators should inspect the remote Vercel project build logs to identify any cloud-side dependency or routing failures.

